

結果測試

1.在 terminal 透過

```
sudo mn --custom=topo.py --topo=mytopo --  
controller=remote,ip=127.0.0.1,port=6653 --switch=ovs,protocols=OpenFlow13
```

建立對應 topology 的 mininet

2.打開新的 terminal 運行 ryu-manager

```
ryu-manager lab2.py
```

3.打開新的 terminal 去做 Restful API 指令的下達

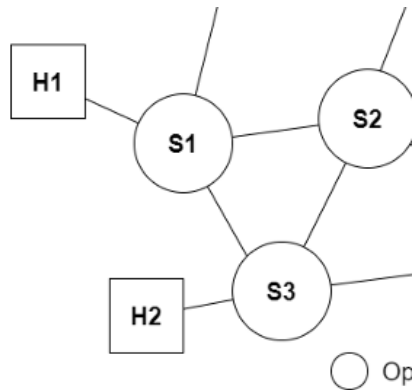
```
curl -X POST -H "Content-Type: application/json" -d '{"src":"h1","dst":"h2"}'  
http://127.0.0.1:8080/v1.0/dualpath/route
```

src 跟 dst 可以根據需要測試的 host 去調整

運行結果

在收到 Restful API 對於起點和終點的設置後就會計算兩條最短路徑，並且設置對應的 flow rule，然後回傳計算的結果

```
cn@cn-VirtualBox:~/lab2$ curl -X POST -H "Content-Type: application/json" -d '{"src": "h1", "dst": "h2", "src_ip": "10.0.0.1", "dst_ip": "10.0.0.2"}' http://10.0.0.1:8080/api/paths
{"status": "success", "message": "Found paths and installed flows", "src": "h1", "dst": "h2", "src_ip": "10.0.0.1", "dst_ip": "10.0.0.2", "path1": [1, 3], "path2": [1, 2, 3]}
```



所以對於 h1 h2 來講最短路徑是 h1-s1-s3-h2，第二條則是 h1-s1-s2-s3-h2。一開始因為連線還沒建立所以 ping 無法成功，但是建立完連線後就可以順利連接，而這時候即使發生 s1-s3 link down 也可以透過第二條路徑繼續連線，直到 s1-s2 也發生 link down 才會導致 h1 h2 之間無法連線。

連線建立後才能
順利連接

```
mininet> h1 ping h2 -c 3
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2050ms
pipe 3
mininet> h1 ping h2 -c 3
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.988 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.065 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.065 ms
```

只有 h1 h2 能連線

```
--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2017ms
rtt min/avg/max/mdev = 0.065/0.372/0.988/0.435 ms
mininet> h1 ping h3 -c 3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable
From 10.0.0.1 icmp_seq=2 Destination Host Unreachable
From 10.0.0.1 icmp_seq=3 Destination Host Unreachable
```

s1 s3 link down 還
是可以透過第二條
路徑連接

```
--- 10.0.0.3 ping statistics ---
3 packets transmitted, 0 received, +3 errors, 100% packet loss, time 2051ms
pipe 3
mininet> link s1 s3 down
mininet> h1 ping h2 -c 3
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.569 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.055 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.066 ms

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2048ms
rtt min/avg/max/mdev = 0.055/0.230/0.569/0.239 ms
mininet> link s1 s2 down
mininet> h1 ping h2 -c 3
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.

--- 10.0.0.2 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2048ms
```

根據 lab2 需要完成的功能，我分成以下幾個部分來實作

1. Restful API 的設置，讓 ryu-manager 在運行的時候，可以透過 API 傳送想運算的路徑是從哪裡開始到哪裡結束
2. two shortest & most-disjoint paths 的運算，針對測試的 topology 計算從 source 到 destination 的最短路徑，以及盡量不與最短路徑重疊的第二最短路徑
3. 將計算出來的路徑所經過的 switch 上面設置對應的 flow rule，讓 source 和 destination 可以順利連接

接下來是各個部分的實作細節

1. Restful API 的設置，讓 ryu-manager 在運行的時候，可以透過 API 傳送想運算的路徑是從哪裡開始到哪裡結束

```
@route('dualpath', REST_API_PREFIX + '/dualpath/route', methods=['POST'])
def set_route(self, req, **kwargs):
    try:
        data = req.json if req.body else {}
        src = data.get('src', None)
        dst = data.get('dst', None)

        if not src or not dst:
            return self._error_response('src or dst missing')

        host_data = self.dualpath_app.topology_data.get('hosts', {})
        if src not in host_data or dst not in host_data:
            return self._error_response('src or dst host not found in topology_data')

        src_ip = host_data[src]['ip']
        dst_ip = host_data[dst]['ip']
        src_dpid = host_data[src]['dpid']
        dst_dpid = host_data[dst]['dpid']
```

route decorator 會將 set_route 方法註冊為一個可以透過 HTTP POST 方法訪問的路由，路由的 URL 為 /v1.0/dualpath/route。

而 set_route function 會根據傳送進來的資料解析其中 host 的資訊，包含起點 src 及終點 dst，然後紀錄各自的 ip 位置，在後續的 flow rule table 設置會使用。如果有發生錯誤則會回傳 error 的資訊”src or dst host not found in topology_data”。

2.two shortest & most-disjoint paths 的運算，針對測試的 topology 計算從 source 到 destination 的最短路徑，以及盡量不與最短路徑重疊的第二最短路徑

```
def compute_shortest_path(self, src, dst):
    graph = self.topology_data['graph']
    dist = {node: float('inf') for node in graph}
    dist[src] = 0
    prev = {node: None for node in graph}
    pq = [(0, src)]
    heapq.heapify(pq)

    while pq:
        current_dist, u = heapq.heappop(pq)
        if u == dst:
            break
        if current_dist > dist[u]:
            continue
        for v, info in graph[u].items():
            w = info['weight']
            new_dist = dist[u] + w
            if new_dist < dist[v]:
                dist[v] = new_dist
                prev[v] = u
                heapq.heappush(pq, (new_dist, v))

    if dist[dst] == float('inf'):
        return None

    path = []
    node = dst
    while node is not None:
        path.append(node)
        node = prev[node]
    path.reverse()
    return path
```

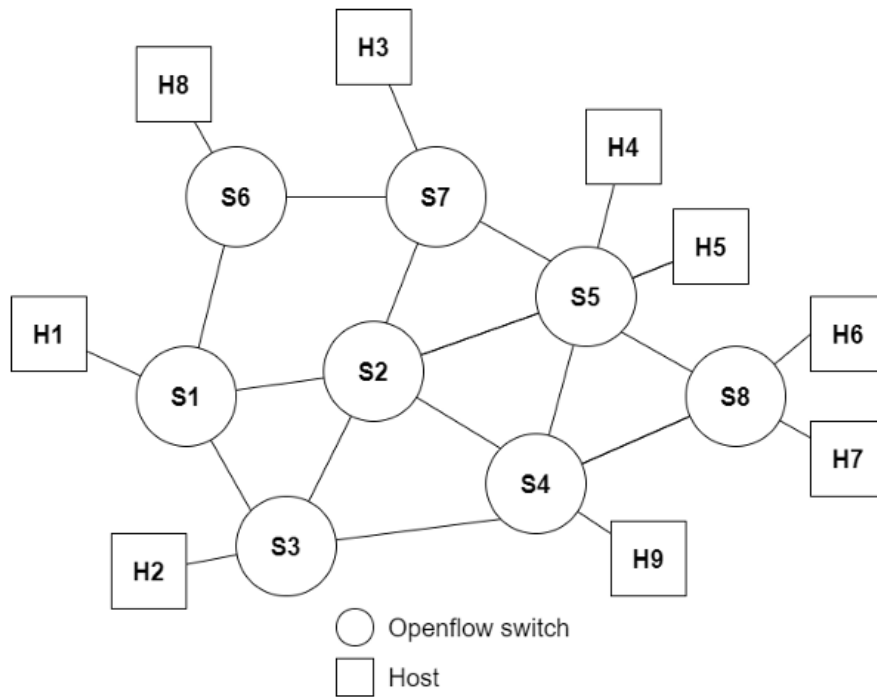
透過 Dijkstra 演算法根據 topology 的設定，計算從 src_dpid 到 dst_dpid 的最短路徑。計算完最短路徑後，需要去提高這條路徑的權重，我這邊是設置原始權重是 1，而如果走過的話會改成 1000，這樣才能盡量避免第二條路徑與最短路徑的重疊。更新完權重之後會再重複一次 Dijkstra 演算法，來計算第二條的路徑。最後還原權重，防止影響到其他 source destination pair 的路徑計算。

```
# 計算第一條最短路徑
path1 = self.dualpath_app.compute_shortest_path(src_dpid, dst_dpid)
if not path1:
    return self._error_response('No path found between src and dst')

# 提高第一條路徑的權重
self.dualpath_app.increase_path_weight(path1, factor=1000)

# 計算第二條路徑
path2 = self.dualpath_app.compute_shortest_path(src_dpid, dst_dpid)

# 還原權重
self.dualpath_app.reset_weights()
```



一些其他 source - destination pair 計算的結果

```
{
  "status": "success",
  "message": "Found paths and installed flows",
  "src": "h7",
  "dst": "h8",
  "src_ip": "10.0.0.7",
  "dst_ip": "10.0.0.8",
  "path1": [
    8,
    5,
    7,
    6
  ],
  "path2": [
    8,
    4,
    2,
    1,
    6
  ]
}

{
  "status": "success",
  "message": "Found paths and installed flows",
  "src": "h3",
  "dst": "h1",
  "src_ip": "10.0.0.3",
  "dst_ip": "10.0.0.1",
  "path1": [
    7,
    2,
    1
  ],
  "path2": [
    7,
    6,
    1
  ]
}
```

```
{
  "status": "success",
  "message": "Found paths and installed flows",
  "src": "h2",
  "dst": "h4",
  "src_ip": "10.0.0.2",
  "dst_ip": "10.0.0.4",
  "path1": [
    3,
    2,
    5
  ],
  "path2": [
    3,
    4,
    5
  ]
}

{
  "status": "success",
  "message": "Found paths and installed flows",
  "src": "h4",
  "dst": "h9",
  "src_ip": "10.0.0.4",
  "dst_ip": "10.0.0.9",
  "path1": [
    5,
    4
  ],
  "path2": [
    5,
    2,
    4
  ]
}
```

3.將計算出來的路徑所經過的 switch 上面設置對應的 flow rule，讓 source 和 destination 可以順利連接

```
def install_flow_for_path(self, src_ip, dst_ip, path):
    # 取得目標 host 的 dp與port
    hosts_info = self.topology_data['hosts']
    dst_dpid = None
    dst_port = None
    for h, info in hosts_info.items():
        if info['ip'] == dst_ip:
            dst_dpid = info['dpid']
            dst_port = info['port']
            break

    graph = self.topology_data['graph']

    for i in range(len(path)):
        dpid = path[i]
        if dpid not in self.datapaths:
            continue
        datapath = self.datapaths[dpid]
        parser = datapath.ofproto_parser
        ofproto = datapath.ofproto

        if i == len(path)-1:
            # 最後一個 switch 連接 host
            out_port = dst_port
        else:
            next_hop = path[i+1]
            out_port = graph[dpid][next_hop]['port']

        # 安裝 IPv4 Flow
        match_ip = parser.OFPMatch(
            eth_type=0x0800,
            ipv4_src=src_ip,
            ipv4_dst=dst_ip
        )
        actions = [parser.OFPACTIONOutput(out_port)]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS, actions)]
        mod_ip = parser.OFPFlowMod(
            datapath=datapath,
            priority=1,
            match=match_ip,
            instructions=inst
        )
        datapath.send_msg(mod_ip)
```

針對計算出來的 path 所經過的 switch 設置從 source 到 destination 的 flow rule table，除了要設置 IPv4 的規則之外，也要處理 ARP 的規則，因為會先透過 ARP 找尋 MAC address，同時也要設置反向的 flow rule 來讓 destination 可以成功回復 ping 給 source。而第二條 path 也是做相同的處理。

```
# 下發 flow entries 到 switch 上 (包含反向)
if path1:
    # 正向
    self.dualpath_app.install_flow_for_path(src_ip, dst_ip, path1)
    # 反向 (直接reverse路徑，並交換src/dst IP)
    reversed_path1 = path1[::-1]
    self.dualpath_app.install_flow_for_path(dst_ip, src_ip, reversed_path1)

if path2:
    self.dualpath_app.install_flow_for_path(src_ip, dst_ip, path2)
    reversed_path2 = path2[::-1]
    self.dualpath_app.install_flow_for_path(dst_ip, src_ip, reversed_path2)
```